# Day 5 exercises

# Day 5 Exercises: Data Manipulation and Analysis

*PhD Course in Integrative Neurosciences - Introduction to Scientific Programming*

## Instructions

These exercises are designed to build your skills progressively. Work through them in order, as later exercises build on concepts from earlier ones.

**For each exercise:**

1. Read the problem statement carefully
2. Try to solve it yourself before looking at hints
3. Test your solution with the provided data
4. Compare your approach with the provided solution
5. Experiment with variations

**Getting help:**

- Use `?function_name` or `help(function_name)` to see documentation
- Check pandas documentation: https://pandas.pydata.org/docs/
- SciPy documentation: https://docs.scipy.org/doc/scipy/
- Ask AI coding assistants for explanations (but try first yourself!)

---

## Part 1: Data Loading and Exploration

### Exercise 1.1: Loading and Inspecting Data

**Scenario:** You have received experimental data from a behavioral neuroscience study examining reaction times in different experimental conditions.

**Tasks:**

a) Create a small synthetic dataset to work with (or use your own dataset):

```
        import pandas as pd
        import numpy as np

        # Set random seed for reproducibility
        np.random.seed(42)

        # Create sample data
        n_subjects = 30
        n_trials_per_condition = 10

        data = {
            'subject_id': np.repeat(range(1, n_subjects + 1),
n_trials_per_condition * 3),
            'condition': np.tile(['control', 'drug_a', 'drug_b'], n_subjects *
n_trials_per_condition),
            'trial': np.tile(range(1, n_trials_per_condition + 1), n_subjects
* 3),
            'reaction_time': np.random.normal(350, 80, n_subjects *
n_trials_per_condition * 3),
            'accuracy': np.random.uniform(0.6, 1.0, n_subjects *
n_trials_per_condition * 3),
            'session': np.random.choice(['morning', 'afternoon'], n_subjects *
n_trials_per_condition * 3)
        }

        df = pd.DataFrame(data)

        # Add some realistic effects
        df.loc[df['condition'] == 'drug_a', 'reaction_time'] += 30
        df.loc[df['condition'] == 'drug_b', 'reaction_time'] -= 20
        df.loc[df['session'] == 'afternoon', 'reaction_time'] += 15

        # Round values
        df['reaction_time'] = df['reaction_time'].round(1)
        df['accuracy'] = df['accuracy'].round(3)
```

b) Perform initial data inspection:

- Display the first 10 rows
- Check the shape of the DataFrame
- List all column names and their data types
- Generate summary statistics for all numerical columns

c) Answer the following questions:

- How many total observations are in the dataset?
- How many unique subjects participated?
- What are the three experimental conditions?
- What is the range of reaction times?

**Solution:** See `solutions/exercise_1_1.py`

---

### Exercise 1.2: Data Quality Assessment

**Scenario:** Before analyzing data, you need to check for data quality issues.

**Tasks:**

a) Introduce some missing values to practice data quality checks:

```
# Introduce missing values randomly
import random
random.seed(42)

# Create a copy to work with
df_missing = df.copy()

# Randomly set some values to NaN
missing_indices_rt = random.sample(range(len(df_missing)), 25)
missing_indices_acc = random.sample(range(len(df_missing)), 30)

df_missing.loc[missing_indices_rt, 'reaction_time'] = np.nan
df_missing.loc[missing_indices_acc, 'accuracy'] = np.nan
```

b) Perform data quality checks:

- Count missing values in each column
- Calculate the percentage of missing data per column
- Identify rows with any missing values
- Find subjects with more than 5% missing data

c) Visualize missing data patterns:

- Create a simple visualization showing which columns have missing data

**Solution:** See `solutions/exercise_1_2.py`

---

# Part 2: Data Selection and Filtering

### Exercise 2.1: Boolean Indexing

**Scenario:** You need to filter the data to focus on specific subsets for analysis.

**Tasks:**

a) Select all trials where:

- Reaction time is less than 300 ms (fast responses)
- Accuracy is greater than 0.9 (high accuracy)
- Both conditions above are met

b) Find all trials from subject 15 in the 'drug_a' condition

c) Select trials from the morning session with reaction times between 300-400 ms

d) Find trials where accuracy is below 0.7 OR reaction time is above 500 ms (potentially problematic trials)

e) Count how many trials meet each of the above criteria

**Solution:** See `solutions/exercise_2_1.py`

---

### Exercise 2.2: .loc[] and .iloc[] Practice

**Scenario:** Practice different methods of selecting data from DataFrames.

**Tasks:**

a) Using `.loc[]`:

- Select rows 0-9 and columns 'subject_id', 'reaction_time', and 'accuracy'
- Select all trials where condition is 'control'
- Select reaction_time for trials with accuracy > 0.85

b) Using `.iloc[]`:

- Select the first 20 rows and first 4 columns
- Select every 10th row (0, 10, 20, ...)
- Select the last 5 rows and last 2 columns

c) Modify data using `.loc[]`:

- Create a new column 'performance_category' with default value 'medium'
- Set 'performance_category' to 'high' where accuracy > 0.9
- Set 'performance_category' to 'low' where accuracy < 0.7

**Solution:** See `solutions/exercise_2_2.py`

---

## Part 3: Data Cleaning

### Exercise 3.1: Handling Missing Data

**Scenario:** You need to handle missing data before analysis. Different strategies are appropriate for different situations.

**Tasks:**

a) Using the `df_missing` DataFrame from Exercise 1.2:

- Count total missing values
- Calculate mean reaction time with and without dropping missing values
- Compare different handling strategies

b) Implement different imputation strategies:

- Drop rows with any missing values
- Fill missing reaction times with the median
- Fill missing reaction times with condition-specific medians
- Use forward fill for missing values

c) Compare the results:

- Calculate summary statistics for each approach
- Determine which approach preserves the most data
- Discuss when each approach would be appropriate

**Solution:** See `solutions/exercise_3_1.py`

---

## Exercise 3.2: Detecting and Handling Outliers

**Scenario:** Outliers can significantly affect statistical analyses. You need to identify and handle them appropriately.

**Tasks:**

a) Introduce some outliers into the data:

```
        # Add outliers to practice detection
        df_outliers = df.copy()

        # Add some extreme reaction times (data entry errors or inattention)
        outlier_indices = np.random.choice(df_outliers.index, 15,
replace=False)
        df_outliers.loc[outlier_indices, 'reaction_time'] =
np.random.uniform(800, 1200, 15)

        # Add some very fast responses (anticipation)
        fast_indices = np.random.choice(df_outliers.index, 10, replace=False)
        df_outliers.loc[fast_indices, 'reaction_time'] =
np.random.uniform(100, 150, 10)
```

b) Detect outliers using two methods:

- Z-score method (assumes normal distribution)
- IQR method (robust, non-parametric)

c) Visualize outliers:

- Create box plots showing outliers
- Create histograms before and after outlier removal

d) Handle outliers:

- Remove outliers completely
- Cap outliers (winsorization)
- Keep outliers but flag them

e) Compare impact on summary statistics

**Solution:** See `solutions/exercise_3_2.py`

---

# Part 4: Data Transformation and Grouping

## Exercise 4.1: Creating New Variables

**Scenario:** You need to create derived variables for analysis.

**Tasks:**

a) Create the following new columns:

- `rt_seconds`: Reaction time in seconds instead of milliseconds
- `efficiency`: Accuracy divided by reaction time in seconds
- `performance_score`: Combined measure (accuracy * 100 - rt_seconds)

b) Categorize continuous variables:

- Create `rt_category`: 'fast' (<300ms), 'medium' (300-450ms), 'slow' (>450ms)
- Create `accuracy_level`: 'low' (<0.7), 'medium' (0.7-0.9), 'high' (>0.9)

c) Create z-scored versions of reaction_time and accuracy

d) Create binary indicators:

- `is_correct`: True if accuracy > 0.5
- `is_drug`: True if condition contains 'drug'

**Solution:** See `solutions/exercise_4_1.py`

---

## Exercise 4.2: Grouping and Aggregation

**Scenario:** Analyze how experimental manipulations affect performance.

**Tasks:**

a) Calculate group means:

- Mean reaction time by condition
- Mean accuracy by condition and session
- Count trials per subject

b) Multiple aggregations:

- For each condition, calculate mean, std, min, max, and count of reaction_time
- For each subject, calculate mean RT and accuracy

c) Custom aggregations:

- Calculate coefficient of variation (CV = std/mean) for RT by condition
- Calculate percentage of accurate trials by subject and condition

d) Using transform:

- Create within-subject z-scores for reaction time
- Calculate percent of subject's mean RT

e) Filtering groups:

- Keep only subjects with >85% overall accuracy
- Keep only conditions with mean RT < 400ms

**Solution:** See `solutions/exercise_4_2.py`

---

# Part 5: Reshaping and Combining Data

## Exercise 5.1: Pivot Tables and Melting

**Scenario:** Restructure data for different analysis needs.

**Tasks:**

a) Create a pivot table:

- Rows: subject_id
- Columns: condition
- Values: mean reaction_time

b) Create a more complex pivot table:

- Multiple aggregation functions (mean and std)
- Add row and column totals (margins)

c) Melt data from wide to long format:

- Create a wide-format dataset with one column per condition
- Melt it back to long format

d) Practical application:

- Calculate the difference in RT between drug and control conditions for each subject

**Solution:** See `solutions/exercise_5_1.py`

---

## Exercise 5.2: Merging and Concatenating

**Scenario:** Combine data from multiple sources.

**Tasks:**

a) Create additional datasets to merge:

```
        # Subject demographics
        demographics = pd.DataFrame({
            'subject_id': range(1, 31),
            'age': np.random.randint(20, 40, 30),
            'sex': np.random.choice(['M', 'F'], 30),
            'handedness': np.random.choice(['R', 'L'], 30, p=[0.9, 0.1])
        })

        # Subject genetic data (only subset of subjects)
        genetics = pd.DataFrame({
            'subject_id': [1, 3, 5, 7, 9, 11, 13, 15, 17, 19],
            'COMT_val158met': np.random.choice(['val/val', 'val/met',
'met/met'], 10),
            'BDNF_val66met': np.random.choice(['val/val', 'val/met',
'met/met'], 10)
        })

        # Additional session from different experiment
        session2_data = df.sample(n=300, random_state=42).copy()
        session2_data['session_num'] = 2
        df_temp = df.copy()
        df_temp['session_num'] = 1
```

b) Perform different types of merges:

- Inner join (only subjects in both datasets)
- Left join (keep all behavioral data, add demographics where available)
- Right join
- Outer join (keep everything)

c) Concatenate data:

- Stack session 1 and session 2 data vertically
- Handle overlapping columns

d) Merge multiple datasets:

- Combine behavioral data with demographics and genetics
- Handle subjects without genetic data

**Solution:** See `solutions/exercise_5_2.py`

---

# Part 6: Statistical Analysis with SciPy

## Exercise 6.1: Hypothesis Testing

**Scenario:** Test whether experimental manipulations significantly affect performance.

**Tasks:**

a) Independent samples t-test:

- Compare reaction times between control and drug_a conditions
- Check assumptions (normality, equal variances)
- Report results in standard format

b) Paired samples t-test:

- Compare baseline vs drug conditions within subjects
- Calculate effect size (Cohen's d)

c) Non-parametric alternative:

- Perform Mann-Whitney U test
- Compare results with t-test

d) ANOVA:

- Compare all three conditions simultaneously
- Perform post-hoc pairwise comparisons if significant

**Solution:** See `solutions/exercise_6_1.py`

---

## Exercise 6.2: Correlation and Distribution Fitting

**Scenario:** Explore relationships between variables and fit distributions to data.

**Tasks:**

a) Correlation analysis:

- Calculate Pearson correlation between age and RT
- Calculate Spearman correlation between age and RT
- Create correlation matrix for all numeric variables
- Visualize with heatmap

b) Fit distributions:

- Fit normal distribution to RT data
- Fit exponential distribution to inter-trial intervals
- Compare goodness of fit

c) Q-Q plots:

- Create Q-Q plot for normality assessment
- Interpret deviations from theoretical line

**Solution:** See `solutions/exercise_6_2.py`

---

# Challenge Exercises

## Challenge 1: Complete Data Analysis Pipeline

**Scenario:** You receive a raw dataset from a collaborator and need to perform a complete analysis from start to finish.

**Task:** Perform a complete analysis pipeline including:

1. Load data and perform initial quality checks
2. Clean data (handle missing values, outliers)
3. Create derived variables
4. Exploratory visualization
5. Statistical testing
6. Create a summary report

**Dataset:** Use the `df` DataFrame we've been working with, but introduce realistic issues (missing data, outliers, etc.)

**Solution:** See `solutions/challenge_1.py`

---

## Challenge 2: Advanced Grouping and Aggregation

**Scenario:** Analyze how drug effects vary by subject characteristics.

**Task:** Determine whether age moderates the drug effect on reaction time.

**Solution:** See `solutions/challenge_2.py`

---

## Challenge 3: Missing Data Analysis

**Scenario:** Determine whether missing data is random or systematic.

**Task:** Analyze missing data patterns and determine if missingness relates to other variables.

**Solution:** See `solutions/challenge_3.py`

---

# Additional Resources

**Documentation:**

- Pandas: https://pandas.pydata.org/docs/
- SciPy.stats: https://docs.scipy.org/doc/scipy/reference/stats.html
- NumPy: https://numpy.org/doc/

**Books:**


- McKinney, W. (2022). *Python for Data Analysis* (3rd ed.)

- VanderPlas, J. (2016). *Python Data Science Handbook*

**Tutorials:**

- Pandas Getting Started: https://pandas.pydata.org/docs/getting_started/
- SciPy Tutorial: https://docs.scipy.org/doc/scipy/tutorial/

**Statistical Guidance:**

- When to use which test: https://stats.idre.ucla.edu/other/mult-pkg/whatstat/
- Effect size interpretation: https://www.psychologicalscience.org/observer/why-you-need-to-report-effect-sizes

---

# Solutions Summary

All exercises have detailed solutions provided in the `solutions/` directory. Work through them systematically:

1. **Part 1-2**: Master data loading, exploration, and selection
2. **Part 3**: Learn thorough data cleaning practices
3. **Part 4**: Understand grouping and transformation
4. **Part 5**: Practice reshaping and merging
5. **Part 6**: Apply statistical testing correctly
6. **Challenges**: Integrate all skills

Remember: Data analysis is iterative. You'll often go back and forth between steps as you discover new insights!

---

*Exercises prepared for CNC-UC Introduction to Scientific Programming*
*University of Coimbra*